Lab 8 Instructions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | October 28, 2026

1 Function: `hard_nms`

Problem Statement

Object detection models frequently predict multiple overlapping bounding boxes for the exact same physical object. Implement a post-processing algorithm (Non-Maximum Suppression) that eliminates duplicate and redundant bounding boxes, retaining only the single highest-confidence detection per entity.

Implementation Instructions

1. **Extract Values:** Separate the input matrix into two components: confidence scores and coordinate tuples (x_1, y_1, x_2, y_2).
2. **Sort by Confidence:** Order the candidate bounding boxes in ascending order according to their detection probability scores.
3. **Limit Candidates:** Retain only the highest-scoring candidate indices up to `candidate_size` to avoid unnecessary computation over low-probability background regions.
4. **Iterative Suppression:** While candidate indices remain:
   - Extract the box associated with the highest remaining score (the final index in the sorted list) and append it to the `picked` collection.
   - Terminate the loop immediately if the total number of picked boxes reaches `top_k` or if only one candidate remains.
   - Compare this designated box against all remaining candidate boxes by calculating their Intersection over Union (IoU):
     - Determine the coordinates of the intersecting rectangle between the active box and each remaining box.
     - Compute the intersection area, clamping negative dimensions to zero:

$$\text{Intersection} = \max(0, x_2^{\min} - x_1^{\max}) \times \max(0, y_2^{\min} - y_1^{\max})$$

     - Compute the combined union area:

$$\text{Union} = \text{Area(Active)} + \text{Area(Remaining)} - \text{Intersection}$$

  - Determine the ratio: $\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$.
- Discard all candidate boxes whose IoU exceeds `iou_threshold`.

5. **Return Filtered Subset:** Return the rows of the original array matching the indices retained in `picked`.

---

2 Function: `detect_faces`

Problem Statement

Construct an end-to-end inference routine that ingests an image file from storage, normalizes it into tensor format, executes forward inference via ONNX Runtime using an ultra-lightweight detection model, applies Non-Maximum Suppression to consolidate outputs, and renders labeled bounding boxes on the original image.

Implementation Instructions

1. **Load and Record Geometry:** Open the target image using Pillow, ensure 3-channel RGB formatting, and record the original width and height for post-inference scaling.
2. **Preprocess Input Tensor:**
   - Resize the image array to match the input specification of the network (320×240).
   - Convert integer pixel intensities to 32-bit floating-point format.
   - Normalize pixel values to the range $[-1.0, 1.0]$ via the transformation:
   $$x_{\text{norm}} = \frac{x - 127.0}{128.0}$$
   - Transpose data from Height-Width-Channel $(\text{H}, \text{W}, \text{C})$ to Channel-Height-Width $(\text{C}, \text{H}, \text{W})$, then prepend an explicit batch dimension: $(1, \text{C}, \text{H}, \text{W})$.
3. **Execute Inference Session:** Instantiate an ONNX Runtime session specifying available execution providers (such as CUDA or CPU) and evaluate the graph using the preprocessed array.
4. **Filter and Suppress Candidates:**
   - Remove the batch dimension from output tensors.
   - Mask out detections whose confidence falls below `conf_threshold`.
   - Pass surviving coordinates and scores to `hard_nms` to eliminate overlapping duplicate boxes.
5. **Annotation and Export:**
   - Scale normalized bounding box coordinates back to full image dimensions by multiplying by original width and height.
   - Draw rectangular outlines and confidence text overlays on the image canvas.
   - Save the annotated graphic to the destination filepath.

3 Function: `non_maximum_suppression` (Edge Detection)

Problem Statement

Raw gradient magnitude maps generated by spatial edge filters (such as Sobel operators) yield broad, blurry contours. Implement directional Non-Maximum Suppression to thin candidate edge ridges down to single-pixel width lines along their local gradient vectors.

Implementation Instructions

1. **Initialize Output Matrix:** Allocate a zero-filled floating-point array matching the dimensional height and width of the gradient magnitude map.
2. **Angle Transformation:** Convert gradient direction matrices from radians to degrees, wrapping angles into the standard half-turn domain $[0°, 180°)$.
3. **Traverse Internal Grid:** Iterate across all pixel coordinates (i, j), excluding the outer boundary to preserve a valid 3×3 sampling window.
4. **Discretize Gradient Orientation:** Quantize the angle at pixel (i, j) into one of four directional sectors:
   - **Horizontal Direction ($0° \leq \theta < 22.5°$ or $157.5° \leq \theta \leq 180°$):** Compare intensity against horizontal neighbors: $(i, j-1)$ and $(i, j+1)$.
   - **Diagonal Direction ($22.5° \leq \theta < 67.5°$):** Compare intensity against diagonal neighbors: $(i+1, j-1)$ and $(i-1, j+1)$.
   - **Vertical Direction ($67.5° \leq \theta < 112.5°$):** Compare intensity against vertical neighbors: $(i-1, j)$ and $(i+1, j)$.
   - **Anti-Diagonal Direction ($112.5° \leq \theta < 157.5°$):** Compare intensity against anti-diagonal neighbors: $(i-1, j-1)$ and $(i+1, j+1)$.
5. **Peak Evaluation:**
   - If the magnitude at (i, j) equals or exceeds both opposing directional neighbors, retain its value in the output array.
   - Otherwise, set the pixel value to zero (suppress non-peak responses).
6. **Return Thinned Array:** Output the resulting thinned magnitude matrix.

---

4 Function: `hysteresis_thresholding`

Problem Statement

Thinned edge maps contain both genuine structural boundaries and spurious noise spikes. Implement dual-threshold hysteresis to segment candidates into strong, weak, and non-edge pixels, preserving weak edge pixels only when they are topologically connected to strong structural edges.

Implementation Instructions

1. **Define Response Constants:** Assign standard visualization values: Strong (255), Weak (50), and Suppressed (0).
2. **Apply Dual Thresholds:**
   - Designate all pixels with magnitude $\geq$ `high_thresh` as strong edges.
   - Designate all pixels with magnitude satisfying `low_thresh` $\leq$ value $<$ `high_thresh` as weak edges.
   - Assign all remaining sub-threshold pixels to zero.
3. **Perform Neighborhood Connectivity Search:**
   - Identify the coordinate positions of all weak pixels.
   - Inspect the 8-connected immediate neighborhood (3×3 window) surrounding each weak candidate.
   - If at least one adjacent pixel is strong, promote the weak pixel to a confirmed strong edge (255).
   - If no adjacent neighbor is strong, suppress the weak pixel to zero.
4. **Return Binary Map:** Return the refined edge mask.

---

5 Function: `canny_scipy`

Problem Statement

Construct an end-to-end implementation of the classical Canny Edge Detection pipeline using only SciPy and Pillow (without OpenCV dependencies), producing a refined single-pixel binary outline of physical boundaries present in an image.

Implementation Instructions

1. **Grayscale Conversion and Noise Filtering:**
   - Load the source image from disk and convert it to single-channel luminance (grayscale).
   - Convert integer pixel representations into 32-bit floating-point numbers.
   - Convolve the array with an isotropic Gaussian kernel (standard deviation $\sigma = 1.4$) to attenuate high-frequency sensor noise.
2. **Compute Directional Derivatives and Magnitudes:**
   - Convolve the smoothed field with horizontal (K_x) and vertical (K_y) Sobel differential kernels:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

- Calculate gradient magnitudes via vector hypotenuse: $M = \sqrt{G_x^2 + G_y^2}$.
- Rescale magnitude values into the 8-bit dynamic interval $[0, 255]$.
- Compute spatial orientation angles using the four-quadrant inverse tangent: $\theta = \text{atan2}(G_y, G_x)$.

3. **Execute Non-Maximum Suppression:** Pass magnitude and directional angle arrays into `non_maximum_suppression` to thin gradients down to one-pixel ridges.

4. **Perform Dual-Threshold Hysteresis:** Process the thinned output with `hysteresis_thresholdin` using specified `low` and `high` limits to isolate valid topological edges.

5. **Export Edge Map:** Convert the resulting unsigned 8-bit array into a Pillow image instance and persist it to the target file destination.